StudHub — Cahier des fonctionnalités

Frontend + Backend | Ce qui a été ajouté et développé

Contexte : frontend fourni au départ — backend et features avancées développés ensuite

1. Contexte

Le projet StudHub a été fourni initialement côté frontend (application Ionic / Angular avec données mock). Le backend NestJS + PostgreSQL, ainsi que plusieurs fonctionnalités frontend avancées (cartes, proximité, favoris, OTP e-mail, etc.), ont été développés et branchés ensuite pour transformer le mock en application complète.

Stack : Frontend Ionic / Angular / Capacitor | Backend NestJS + Prisma + PostgreSQL + Redis | Auth JWT | Upload local | Chat WebSocket.

2. Fonctionnalités Frontend ajoutées / branchées

Cartographie & proximité

Carte de localisation (Maps) [FRONT]
Sur la fiche détail d'un bien ou d'un produit : carte intégrée (Google Maps embed) positionnée via géocodage OpenStreetMap / Nominatim à partir de la ville ou de l'adresse. Bouton pour ouvrir Google Maps en externe.

Annonces à proximité (Nearby) [FRONT]
Page /nearby accessible depuis le détail immo ou produit. Affiche les autres annonces de la même ville (hors annonce courante), avec chargement API puis fallback local.

Engagement & social

Favoris (biens, produits, services) [FRONT]
Page Favoris + bascule favori sur les fiches. Branchée sur l'API (PropertyFavorite, ProductFavorite, ProviderFavorite).

Likes & compteurs de vues [FRONT]
Like / unlike sur logements et produits ; compteur de vues uniques affiché sur les fiches détail.

Commentaires du feed [FRONT]
Interface pour lire et publier des commentaires sur les posts du réseau social étudiant.

Avis / reviews [FRONT]
Affichage et dépôt d'avis notés sur biens, produits et prestataires via l'API Reviews.

Suivre un agent (Follow) [FRONT]
Depuis la fiche bien / profil agent : s'abonner pour recevoir les alertes de nouvelles publications.

Auth, médias & UX

OTP inscription par e-mail [FRONT]
Parcours d'inscription avec envoi / vérification d'un code OTP 6 chiffres par e-mail (priorité e-mail plutôt que SMS pour le moment).

Upload photos marketplace / annonces [FRONT]
Sélection et upload de photos via UploadService vers le backend ; affichage via /uploads.

Galerie photos [FRONT]
Composant galerie pour parcourir les images d'une annonce en détail.

Menus étudiant / agent [FRONT]
Menus latéraux (profil, messages, favoris, publier, thème, langue, déconnexion).

Notifications push (mobile) [FRONT]
Service Capacitor pour enregistrer le token FCM auprès du backend.

Profil agent [FRONT]
Page dédiée au profil public d'un agent immobilier.

Chat avancé [FRONT]
Conversations, message de bienvenue système, notes vocales / overlay d'appel (composants partagés).

3. Backend complet (NestJS)

API REST versionnée /api/v1, documentation Swagger sur /docs, fichiers statiques /uploads, monitoring /v1/health. Sécurité : Helmet, CORS (web + LAN + Capacitor), rate-limit, ValidationPipe, JWT + rôles.

Modules métier

Auth [BACK]
Inscription / connexion étudiant & agent, OTP e-mail (send / verify), refresh token, logout, mot de passe oublié, endpoint /me.

Users [BACK]
Lecture et mise à jour du profil étudiant.

Agents [BACK]
Profil public agent + follow / unfollow / follow-status pour les alertes de publication.

Properties (Immobilier) [BACK]
Liste filtrée, CRUD agent, détail avec vue unique, favori, like. Route agent/me pour le dashboard agent.

Products (Marketplace) [BACK]
CRUD produits, filtres (ville, catégorie…), like, favoris (toggle + liste favorites/mine), vues uniques.

Providers (StudServices) [BACK]
Prestataires de services étudiants : liste, création / mise à jour, favoris.

Posts (Feed) [BACK]
Publication de posts, like, liste et création de commentaires.

Reviews [BACK]
Lister et créer des avis notés sur PROPERTY / PRODUCT / PROVIDER.

Chat [BACK]
Conversations, messages, marquage lu, conversation système de bienvenue, gateway WebSocket temps réel.

Push [BACK]
Enregistrement / suppression des tokens appareils FCM ; envoi (ou simulation) de notifications.

Upload [BACK]
Upload multipart d'images stockées localement et servies sous /uploads.

Health [BACK]
Statut API + PostgreSQL + Redis pour le monitoring.

Services transverses

OTP + Email + SMS Twilio [BACK]
Génération / vérification de codes OTP (Redis). Envoi e-mail (nodemailer) pour l'inscription ; SMS Twilio disponible.

Guards JWT / rôles / JWT optionnel [BACK]
Protection des routes selon le rôle (STUDENT, AGENT, ADMIN). JWT optionnel pour les vues anonymes ou connectées.

Migrations BDD ajoutées après le socle

system_welcome_chat [BDD]
Conversations SYSTEM non répondables + messages isSystem.

agent_follow [BDD]
Table AgentFollow (abonnés aux alertes agent).

property_product_likes_views [BDD]
Likes biens / produits + compteurs likesCount / viewsCount.

listing_unique_views [BDD]
Tables PropertyView / ProductView (1 vue / utilisateur ou appareil).

product_provider_favorites [BDD]
Favoris marketplace et StudServices.

4. Synthèse — qui a fourni quoi

Frontend de base (UI Ionic, pages, navigation, mock data) : fourni au démarrage du projet.

Apports développés ensuite :

- Backend NestJS complet (API, auth, modules métier, Prisma, PostgreSQL, Redis)
- Cartes de localisation (Maps embed + ouverture externe)
- Annonces à proximité (Nearby) filtrées par ville
- Favoris produits / providers + page Favoris branchée API
- Likes et vues uniques (front + back)
- Follow agents
- OTP inscription par e-mail
- Upload photos réel (marketplace / annonces)
- Commentaires feed & avis sur fiches détail
- Push notifications, chat système welcome, galerie photos

5. Accès techniques

Frontend : http://localhost:4200

Backend API : http://localhost:3000/api/v1

Swagger : http://localhost:3000/docs

Health : http://localhost:3000/v1/health

Uploads : http://localhost:3000/uploads/...

Document généré pour présenter l'ensemble des fonctionnalités ajoutées côté frontend et backend StudHub. Date : 29 septembre 2026.